

Как у меня теперь организована работа с Cursor/Codex

Я внедрил в проект простой workflow, чтобы агент работал не хаотично, а по понятным правилам.

1. Что появилось в проекте

В корне проекта есть `AGENTS.md`.

Это постоянная инструкция для Cursor/Codex. Там описано:

- как агент должен анализировать задачу;
- где хранить описание задачи;
- где хранить план;
- когда можно писать код;
- когда нужно ждать моего одобрения;
- когда обновлять проектную wiki.

Сам текст задачи больше не пишется в `AGENTS.md`.

Вся рабочая информация хранится в папке `.ai/`:

```
.ai/  
  tasks/  
  knowledge/  
    raw/  
    wiki/
```

2. Для чего нужна `.ai/tasks/`

`.ai/tasks/` — это место для конкретных задач.

Например:

```
.ai/tasks/WUI-4864/  
  case.md  
  implementation_plan_1.md
```

`case.md` — описание задачи простым языком: что нужно сделать, какие есть уточнения, ограничения и ссылки.

`implementation_plan_1.md` — пошаговый план реализации.

Главное правило: один шаг плана — один будущий коммит.

3. Как проходит работа по задаче

Последовательность такая:

1. Я даю Cursor задачу
2. Cursor читает AGENTS.md
3. Cursor создает или обновляет case.md
4. Cursor создает implementation_plan_N.md
5. Я читаю план
6. Если всё ок — пишу "одобряю" или "начинай"
7. Cursor делает только первый шаг
8. Я проверяю изменения
9. Я коммичу
10. После коммита Cursor переходит к следующему шагу

То есть агент не делает всё сразу. Он работает контролируемо: шаг → review → commit → следующий шаг.

4. Для чего нужна `.ai/knowledge/raw/`

`raw/` — это папка для исходных материалов.

Туда можно класть:

- старые спецификации;
- загрузки из Jira;
- скриншоты;
- заметки по референсному ПО;
- описания текущей реализации;
- технические исследования.

Например:

```
.ai/knowledge/raw/legacy/SPEC.md
```

Важно: `raw/` — это не финальная правда. Это просто источники, которые агент может использовать как контекст.

5. Для чего нужна `.ai/knowledge/wiki/`

`wiki/` — это проектная память.

Туда попадает не каждая задача, а только стабильные знания:

- как работает доменная логика;

- как устроены звонки;
- как работают завешенные линии;
- как работают устройства;
- как устроены статусы;
- как работает Electron / IPC / preload;
- как устроены SIP/WebRTC-интеграции;
- какие архитектурные решения уже приняты.

Пример:

```
.ai/knowledge/wiki/domain/quick-call-panel.md
.ai/knowledge/wiki/domain/statuses.md
.ai/knowledge/wiki/frontend/stores.md
.ai/knowledge/wiki/backend-integration/sip-webrtc.md
```

6. Кто ведет wiki

Wiki ведет сам Cursor/Codex, но не автоматически в фоне.

Он обновляет её только во время выполнения задачи, если в процессе появилось важное долгоживущее знание.

Например, если мы разобрали, как теперь должна работать панель быстрого вызова, это стоит записать в wiki.

Если просто поправили CSS-отступ — wiki не трогаем.

7. Зачем это нужно

Главная цель — не потерять проектные знания.

Раньше знания были размазаны по чатам, Jira, коду и личной памяти.

Теперь схема такая:

```
AGENTS.md
  правила работы агента

.ai/tasks/
  текущие задачи и планы

.ai/knowledge/raw/
  исходные материалы

.ai/knowledge/wiki/
  актуальное понимание проекта
```

Это помогает:

- быстрее входить в задачу;
- меньше объяснять агенту одно и то же;
- фиксировать решения;
- не терять контекст между задачами;
- делать изменения маленькими и проверяемыми;
- проще проводить review.

8. Главное правило

Cursor не должен просто “писать код”.

Он должен работать так:

понять задачу → составить план → дождаться одобрения → сделать один шаг → дождаться коммита → продолжить

А если в задаче появилось знание, которое пригодится дальше, он должен обновить `.ai/knowledge/wiki/`.